Python Projects

Build a Portfolio of Real Python Applications

■ LEARNING OUTCOME

By the end of this module you will have 4 complete, portfolio-ready Python projects: a CLI Task Manager, Web Scraper, Data Analyser, and REST API client -- with clean code, logging, error handling, and documentation.

■ Skills You Will Gain
• Build a command-line Task Manager with file persistence
• Create a web scraper with requests and BeautifulSoup
• Analyse CSV data with pandas and visualise with matplotlib
• Build a REST API client with retry logic and caching
• Write clean, tested, documented production Python
• Structure a Python project professionally

■ **Four well-built Python projects on GitHub get more data/backend job interviews than any certificate. Projects prove you can build real things that work reliably.**

PROJECT 1

CLI Task Manager

```
"""task_manager.py -- Simple CLI task manager with JSON persistence""" import json import sys from pathlib import Path from datetime import datetime from dataclasses import dataclass, asdict, field from typing import Optional @dataclass class Task: id: int title: str done: bool = False priority: str = "medium" created_at: str = field(default_factory=lambda: datetime.now().isoformat()) due_date: Optional[str] = None class TaskManager: def __init__(self, filepath="tasks.json"): self.filepath = Path(filepath) self.tasks: list[Task] = [] self._next_id = 1 self._load() def _load(self): if self.filepath.exists(): data = json.loads(self.filepath.read_text()) self.tasks = [Task(**t) for t in data.get("tasks",[])] self._next_id = data.get("next_id",1) def _save(self): data = {"tasks":[asdict(t) for t in self.tasks],"next_id":self._next_id} self.filepath.write_text(json.dumps(data,indent=2)) def add(self, title: str, priority="medium", due=None) -> Task: task = Task(id=self._next_id, title=title, priority=priority, due_date=due) self.tasks.append(task) self._next_id += 1 self._save() print(f"Added [{task.id}] {title}") return task def complete(self, task_id: int) -> bool: task = next((t for t in self.tasks if t.id==task_id), None) if not task: print(f"Task {task_id} not found") return False task.done = True self._save() print(f"Completed [{task_id}] {task.title}") return True def delete(self, task_id: int) -> bool: before = len(self.tasks) self.tasks = [t for t in self.tasks if t.id != task_id] self._save() return len(self.tasks) < before def list_tasks(self, show_done=False): tasks = self.tasks if show_done else [t for t in self.tasks if not t.done] if not tasks: print("No tasks!") return priority_order = {"high":0,"medium":1,"low":2} for t in sorted(tasks,key=lambda t:priority_order.get(t.priority,1)): status = "done" if t.done else t.priority print(f"[{t.id}] [{status:6}] {t.title}") if __name__ == "__main__": mgr = TaskManager() cmd = sys.argv[1] if len(sys.argv)>1 else "list" if cmd=="add" and len(sys.argv)>2: mgr.add(" ".join(sys.argv[2:])) elif cmd=="done" and len(sys.argv)>2: mgr.complete(int(sys.argv[2])) elif cmd=="del" and len(sys.argv)>2: mgr.delete(int(sys.argv[2])) else: mgr.list_tasks()
```

PROJECT 2

Web Scraper

```
"""scraper.py -- Scrape quotes from quotes.toscrape.com""" import requests import json import time import logging from bs4 import BeautifulSoup from pathlib import Path logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s") logger = logging.getLogger(__name__) def scrape_quotes(base_url="http://quotes.toscrape.com", max_pages=5): quotes = [] session = requests.Session() session.headers["User-Agent"] = "Mozilla/5.0 QuoteScraper/1.0" for page in range(1, max_pages+1): url = f"{base_url}/page/{page}/" logger.info(f"Scraping page {page}: {url}") try: resp = session.get(url, timeout=10) resp.raise_for_status() # raise on 4xx/5xx except requests.RequestException as e: logger.error(f"Failed page {page}: {e}") break soup = BeautifulSoup(resp.text, "html.parser") quote_divs = soup.select("div.quote") if not quote_divs: logger.info("No more quotes -- done") break for div in quote_divs: quote = { "text": div.select_one("span.text").get_text(strip=True), "author": div.select_one("small.author").get_text(strip=True), "tags": [t.get_text() for t in div.select("a.tag")], "page": page } quotes.append(quote) logger.info(f"Found {len(quote_divs)} quotes on page {page}") time.sleep(1) # be polite -- 1 second between requests return quotes if __name__ == "__main__": quotes = scrape_quotes(max_pages=3) logger.info(f"Total quotes scraped: {len(quotes)}") # Save to JSON Path("quotes.json").write_text(json.dumps(quotes, indent=2, ensure_ascii=False)) logger.info("Saved to quotes.json") # Stats from collections import Counter all_tags = [tag for q in quotes for tag in q["tags"]] top_tags = Counter(all_tags).most_common(5) print("\nTop 5 tags:", top_tags)
```

PROJECT 3

Data Analyser with pandas

```
"""analyser.py -- Sales data analysis with pandas and matplotlib""" import pandas as pd import matplotlib.pyplot as plt from pathlib import Path def load_data(filepath: str) -> pd.DataFrame: """Load CSV with validation.""" df = pd.read_csv(filepath, parse_dates=["order_date"]) required = ["order_id","customer_id","order_date","amount","city","category"] missing = [c for c in required if c not in df.columns] if missing: raise ValueError(f"Missing columns: {missing}") return df def analyse_sales(df: pd.DataFrame) -> dict: """Generate sales analytics report.""" df = df.dropna(subset=["amount"]).copy() df["month"] = df["order_date"].dt.to_period("M") df["year"] = df["order_date"].dt.year # Monthly revenue monthly = df.groupby("month")["amount"].agg(["sum","count","mean"]).reset_index() monthly.columns = ["month","revenue","orders","avg_order"] monthly["mom_growth"] = monthly["revenue"].pct_change() * 100 # Revenue by city by_city = df.groupby("city")["amount"].agg(revenue="sum",orders="count").reset_index() by_city["pct"] = 100 * by_city["revenue"] / by_city["revenue"].sum() by_city = by_city.sort_values("revenue",ascending=False) # Category performance by_cat = df.groupby("category").agg( revenue=("amount","sum"), orders=("order_id","count"), avg_order=("amount","mean") ).reset_index().sort_values("revenue",ascending=False) # Top customers top_customers = df.groupby("customer_id")["amount"].sum()\ .nlargest(10).reset_index() return { "total_revenue": df["amount"].sum(), "total_orders": len(df), "avg_order_value": df["amount"].mean(), "monthly": monthly, "by_city": by_city, "by_cat": by_cat, "top_cust": top_customers } def plot_dashboard(results: dict): """Create 4-panel dashboard.""" fig, axes = plt.subplots(2, 2, figsize=(14,10)) fig.suptitle("Sales Analytics Dashboard", fontsize=16, fontweight="bold") m = results["monthly"] axes[0,0].bar([str(x) for x in m["month"]], m["revenue"], color="#3776AB") axes[0,0].set_title("Monthly Revenue"); axes[0,0].tick_params(axis="x",rotation=45) c = results["by_city"].head(8) axes[0,1].barh(c["city"], c["revenue"], color="#FFD43B") axes[0,1].set_title("Revenue by City") cat = results["by_cat"] axes[1,0].pie(cat["revenue"],labels=cat["category"],autopct="%1.1f%%") axes[1,0].set_title("Revenue by Category") axes[1,1].plot([str(x) for x in m["month"]],m["mom_growth"].fillna(0),marker="o",color="#06D6A0") axes[1,1].axhline(0,color="red",linestyle="--",alpha=0.5) axes[1,1].set_title("MoM Growth %"); axes[1,1].tick_params(axis="x",rotation=45) plt.tight_layout() plt.savefig("dashboard.png",dpi=150,bbox_inches="tight") print("Saved: dashboard.png")
```

PROJECT 4

REST API Client

```
"""api_client.py -- Reusable REST API client with retry, caching, auth""" import requests import time import json import logging from functools import lru_cache from typing import Optional, Any logger = logging.getLogger(__name__) class APIClient: def __init__(self, base_url: str, api_key: Optional[str]=None, timeout=30): self.base_url = base_url.rstrip("/") self.timeout = timeout self.session = requests.Session() if api_key: self.session.headers["Authorization"] = f"Bearer {api_key}" self.session.headers["Content-Type"] = "application/json" def _request(self, method, endpoint, max_retries=3, **kwargs) -> dict: url = f"{self.base_url}{endpoint}" last_error = None for attempt in range(max_retries): try: resp = self.session.request(method,url,timeout=self.timeout,**kwargs) resp.raise_for_status() return resp.json() if resp.content else {} except requests.HTTPError as e: if resp.status_code in (400,401,403,404): # client errors -- don't retry logger.error(f"Client error {resp.status_code} for {url}") raise last_error = e wait = 2 ** attempt # exponential backoff: 1s, 2s, 4s logger.warning(f"Attempt {attempt+1} failed, retrying in {wait}s: {e}") time.sleep(wait) except requests.ConnectionError as e: last_error = e time.sleep(2**attempt) raise RuntimeError(f"All {max_retries} attempts failed") from last_error def get(self, endpoint, params=None): return self._request("GET",endpoint,params=params) def post(self, endpoint, data): return self._request("POST",endpoint,json=data) def put(self, endpoint, data): return self._request("PUT",endpoint,json=data) def delete(self, endpoint): return self._request("DELETE",endpoint) @lru_cache(maxsize=128) def get_cached(self, endpoint: str) -> str: """Cached GET -- same endpoint returns cached result.""" return json.dumps(self.get(endpoint)) # Usage with GitHub API github = APIClient("https://api.github.com") user = github.get("/users/python") repos = github.get("/orgs/python/repos",params={"per_page":10,"sort":"stars"}) print(f"{user['name']}: {user['public_repos']} repos") print([r["name"] for r in repos[:5]])
```

SECTION 5

Project Structure & Interview Prep

1	**Project Structure** src/ package, tests/, docs/, .env for secrets, pyproject.toml or setup.py
2	**README.md** Problem solved, tech stack, setup steps, usage examples, screenshots
3	**Requirements** requirements.txt with pinned versions, requirements-dev.txt for dev tools
4	**Tests** pytest tests/ with at least happy path + error cases per function
5	**Linting** black (formatter), flake8 (linter), mypy (type checker) -- all passing
6	**Git Hygiene** Meaningful commits, .gitignore (venv/, __pycache__, .env), GitHub repo
7	**Documentation** Docstrings on all public functions/classes -- Google or NumPy style
8	**Deploy** GitHub Actions CI, Docker for portability, README badge showing tests pass

Interview Topic	What to Prepare
List vs tuple	Mutability, use cases, performance
dict comprehension	Build from transformation in one line
Generators	yield, memory efficiency, infinite sequences
Decorators	Write timer, cache, retry from scratch
Context managers	Write one with contextmanager
OOP design	Design Bank, Library, or Parking Lot system
Exception handling	Custom exceptions, re-raise, logging
File processing	Read large CSV line by line efficiently
Regex	Email validation, log parsing, data extraction
API integration	Retry logic, auth headers, pagination

■ PRACTICE Q & A

Q1. How do you read a very large file without loading it all into memory?

> ***A:*** *Iterate line by line: 'for line in f:' reads one line at a time into memory. Or use f.read(chunk_size) in a while loop. Never use f.read() or f.readlines() on large files as they load everything at once.*

Q2. What is the difference between deepcopy and copy?

> ***A:*** *copy.copy() creates a shallow copy -- top-level is copied but nested objects still share references. copy.deepcopy() recursively copies everything -- fully independent. Use deepcopy when you need truly independent nested structures.*

Q3. How do you make a Python class iterable?

> ***A:*** *Implement __iter__ (returns self or an iterator object) and __next__ (returns next value, raises StopIteration when done). Or use __iter__ returning a generator.*

Q4. What is the GIL and how does it affect threading?

> ***A:*** *The Global Interpreter Lock allows only ONE Python thread to execute at a time. CPU-bound tasks don't benefit from threading -- use multiprocessing instead. I/O-bound tasks (network, file) do benefit -- threads release GIL during I/O.*

Q5. How would you structure a Python project professionally?

> ***A:*** *src/package_name/ for source code, tests/ for pytest tests, docs/ for documentation, pyproject.toml for build config, .env for secrets (never commit), requirements.txt for dependencies, README.md with setup instructions and usage.*

■ Python Projects Cheat Sheet	
`@dataclass`	Auto-generate __init__,__repr__,__eq__
`from pathlib import Path`	Cross-platform file paths
`Path.read_text() / write_text()`	Simple file IO
`requests.Session()`	Reuse connection for multiple requests
`resp.raise_for_status()`	Raise on HTTP error status
`time.sleep(2**attempt)`	Exponential backoff for retries
`@lru_cache(maxsize=128)`	Cache API/function results
`logging.getLogger(__name__)`	Module-level logger
`json.dumps(data,indent=2)`	Pretty-print JSON
`pd.read_csv(file,parse_dates)`	Load CSV with date parsing
`df.groupby('col').agg(...)`	Pandas groupby aggregation
`pytest tests/`	Run all tests